- *API rate limiting and upload quotas*—Implementing rate limits and quotas helps prevent abuse and denial-of-service attacks by restricting excessive or malicious upload attempts.
- *Malware scanning*—Integrating malware detection tools for uploaded files is essential to prevent the storage of infected or malicious content that could compromise the system.
- *Background processing*—Offloading heavy tasks like document parsing and vector embedding to background workers or serverless functions improves responsiveness, scalability, and fault tolerance.
- *Encrypted storage*—Storing documents and their vector embeddings in encrypted form protects data at rest, reducing the risk of unauthorized access. Alternatively, documents can be securely discarded after processing if retention is not required.

11.2.3 API design and URL structure to minimize information exposure

You may have noticed that our application employs multiple API routes to facilitate backend operations. Designing these API endpoints to expose minimal sensitive information while maintaining clear, intuitive, and RESTful URL structures is vital for both security and maintainability.

We organized our API routes hierarchically under paths like `/api/knowledgebase/[knowledgebaseId]` and `/api/chat/[knowledgebaseId]` to clearly scope operations within the context of specific knowledge bases. Every data access is gated by authentication to prevent unauthorized usage. Additionally, we used secure ID generation methods (such as UUIDs) to avoid predictable or guessable identifiers.

However, there is room for improvement in API security and documentation. For instance, adopting a formal API specification standard like OpenAPI would enhance clarity, facilitate automated documentation, and improve client–server contract enforcement.

Furthermore, integrating a dedicated API gateway with features such as centralized authentication, rate limiting, throttling, and logging would offload these concerns from the main application. This separation helps keep the core codebase lean, easier to maintain, and more secure.

These architectural and operational considerations require careful evaluation and proactive planning when building feature-rich web applications. Neglecting them can result in bloated, fragile, and difficult-to-secure codebases that make future development difficult and expose vulnerabilities.

Since our example RAG web application is primarily intended for educational purposes, we have opted for a minimal viable product approach that delivers core functionality within a single, unified codebase. This design keeps the project simple and accessible for learning and experimentation. But as you prepare to bring your application into production and scale it to support more users and workloads, it becomes essential to consider decomposing the application into dedicated, modular services.